



程序设计原理

模板(Templates)与向量(Vector)

任课教师：喻 梅
于瑞国
王建荣
李雪威
赵满坤



Objectives

- To know the motivation and benefits of templates.
- To define a template function with type parameters.
- To develop a generic sort function using templates.
- To develop generic classes using class.
- To use the C++ vector class as a resizable array.



Templates Basics



Templates functions and classes enable programs to work on many different data types without being rewritten for each one.

Suppose you want to find the maximum of two integers, two doubles, two characters, and two strings. You might write four overloaded functions as follows:

```
int maxValue(int a, int b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```

```
double maxValue(double a, double b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```

```
char maxValue(char a, char b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```

```
string maxValue(string a, string b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```



Templates Basics

These four functions are almost identical, except that each uses a different type.

```
maxValue(int a, int b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```

```
maxValue(char a, char b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```

```
maxValue(double a, double b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```

```
maxValue(short a, short b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```



Templates Basics

You could simply define one function with a generic type as follows:

```
GenericType maxValue(GenericType a, GenericType b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```

To define a template function for finding a maximum value between two values of a generic type:

before a function →

```
template<typename T>  
T maxValue(T a, T b) {  
    if (a > b)  
        return a;  
    else  
        return b;  
}
```



Templates Basics

The definition for the function template begins with the keyword `template` followed by a list of parameters. Each parameter must be preceded by the interchangeable keyword `typename` or `class` in the form `<typename typeParameter>` or `<class typeParameter>`.

Diagram illustrating the syntax of a function template definition:

```
template<typename T>
T maxValue(T a, T b) {
    if (a > b)
        return a;
    else
        return b;
}
```

Annotations:

- Template prefix (before a function) points to `template`.
- a list of parameters points to `<typename T>`.
- `typename T` Or `class T` points to `typename T`.
- function points to the function body (the code inside the curly braces).

Note: Occasionally, a template function may have more than one parameter. In this case, place the parameters together inside the brackets, separated by commas, such as:

`<typename T1, typename T2, typename T3>`



Templates Basics

Template
prefix
(before a
function)

```
template<typename T>
T maxValue(T a, T b) {
    if (a > b) return a;
    else return b;
}

int main(){
    cout << "Maximum between 1 and 3 is "
    << maxValue(1, 3) << endl;
    cout << "Maximum between 1.5 and 0.3 is "
    << maxValue(1.5, 0.3) << endl;
    cout << "Maximum between 'A' and 'N' is "
    << maxValue('A', 'N') << endl;
    cout << "Maximum between \"NBC\" and \"ABC\" is "
    << maxValue(string("NBC"), string("ABC")) << endl;
    return 0;
}
```

```
Maximum between 1 and 3 is 3
Maximum between 1.5 and 0.3 is 1.5
Maximum between 'A' and 'N' is N
Maximum between "NBC" and "ABC" is NBC
```



Example: A Generic Sort



Key
Point

Define a generic selection sort function.

Instead of writing several overloaded sort functions, you can define just one template function that works for any type.

```
void displayArray(int[], int );

void mySwap(int& a, int& b) {...}

void selectionSort(int list[], int size) {...}

void displayArray(int list[], int size) {...}

int main() {
    int list[] = {2, 9, 5, 4, 8, 1, 6};
    selectionSort(list, 7);
    return 0;
}
```




Class Templates



Key
Point

You can define generic types for a class.

1. Combine class definition and class implementation into one file.
2. For definition:
 - a. Place the template prefix before the class definition.
 - b. The type parameter can be used in the class just like any regular data type.
3. For implementation:
 - a. Place the template prefix before the constructor and function header in the implementation and use the type parameter (T) to replace the regular data type in the function.
 - b. Class name before the scope resolution operator :: is `ClassName<T>`
template prefix: `template<typename T>`



Class Templates

1. Combine class definition and class implementation into one file.
2. For definition:
 - a. Place the template prefix before the class definition.
 - b. The type parameter can be used in the class just like any regular data type.
3. For implementation:
 - a. Place the template prefix before the constructor and function header in the implementation.
 - b. Class name before the scope resolution operator `::` is `ClassName<T>`

Stack.h

```
//Definition
1  template<typename T>
2  class Stack{
3  public:
4      Stack();
5      bool isEmpty() const;
6      T peek() const;
7      void push(T value);
8      T pop();
9      int getSize() const;
10     void display() const;
11
12 private:
13     T elements[100];
14     int size;
15 };

//Implementation
...
```



Class Templates

1. Combine class definition and class implementation into one file.
2. For definition:
 - a. Place the template prefix before the class definition.
 - b. The type parameter can be used in the class just like any regular data type.
3. For implementation:
 - a. Place the template prefix before the constructor and function header in the implementation.
 - b. Class name before the scope resolution operator `::` is `ClassName<T>`

Stack.h

```
//Definition
1  template<typename T>
  ...
3  ...
15 };
16

//Implementation
18 template<typename T>
19 Stack<T>::Stack(){
20     size = 0;
21 }
21 template<typename T>
21 T Stack<T>::peek() const {
22     return elements[size-1];
23 }
24 template<typename T>
25 void Stack<T>::push(T value){
26     elements[size++] = value;
27 }
28 ...
...
```



The C++ vector Class



C++ contains a generic **vector** class for storing a list of objects.

```
vector<elementType> vectorName;
```

```
vector<int> intVector;  
vector<string> stringVector;
```

```
#include <vector>  
using namespace std;
```

```
int main() {  
    vector<int> intVector;  
    for (int i = 0; i < 10; i++)  
        intVector.push_back(i + 1);  
  
    intVector.pop_back(); // Remove the last element  
  
    return 0;  
}
```



程序设计原理

Standard Template Libraries

Instructor: Mei Yu

Ruiguo Yu

Jianrong Wang

Xuwei Li

Xiang Ying



Objectives

- **Introduction to Standard Template Library (STL)**
- **Introduction to Containers**
 - Templated data structure
 - vector, list, deque; set, multiset, map, multimap; stack, queue, priority_queue
- **Introduction to Iterators**
 - Access the elements of STL containers
- **Introduction to Algorithms**
 - Program with many STL algorithms
 - equal, size, find, remove, replace, min, max, swap, basic searching, sorting algorithms



Introduction to the Standard Template Library (STL)

- The **Standard Template Library (STL)** defines powerful, **template-based, reusable components** that implement many common **data structures** and **algorithms** used to process those data structures.

algorithms + data structures = programs



Introduction to the Standard Template Library (STL)

- The **Standard Template Library (STL)** defines powerful, template-based, reusable components.
- Implement many common data structures and algorithms used to process those data structures.
- The STL was conceived and designed for performance and flexibility.
- STL has three key components
 - **containers** (popular templated data structures)
 - **iterators** (to access the elements of STL containers)
 - **algorithms** (searching, sorting, comparing etc)



Advantage of STL

- **Data structures.**
 - linked lists, queues, stacks and trees.
 - objects are linked together with pointers.
- **Pointer-based code is complex**
 - the slightest omission or oversight can lead to serious memory-access violations and memory-leak errors with no compiler complaints.
- **Implementing additional data structures, such as deques, priority queues, sets and maps, requires substantial extra work.**
- **An advantage of the STL is that you can reuse the STL *containers, iterators* and *algorithms***
 - implement common data structures and manipulations project-wide.



STL Containers

- Each STL container has associated member functions.
 - A subset of these member functions is defined in all STL containers.
- Example of STL containers
 - **vector** (a dynamically resizable array)
 - **list** (a doubly linked list)
 - **deque** (a double-ended queue, pronounced “deck”).
 - Double-ended queues are sequence containers with dynamic sizes that can be expanded or contracted on both ends (either its front or its back).
 - individual elements are accessed directly through random access iterators



STL Iterators

- **STL iterators**
 - properties similar to those of pointers
 - used by programs to manipulate the STL-container elements.
- **Standard arrays can be manipulated by STL algorithms**
 - using standard pointers as iterators.
- **Manipulating containers with iterators is convenient**
 - provides tremendous expressive power combined with STL algorithms
 - reduce many lines of code to a single statement.



STL Algorithms

- **STL algorithms are functions that perform common data manipulations**
 - **searching, sorting and comparing elements** (or entire containers) etc.
- **Each algorithm has minimum requirements for the types of iterators that can be used with it.**
- **Each first-class container supports specific iterator types, some more powerful than others.**
- **A container's supported iterator type determines whether the container can be used with a specific algorithm.**



Containers

- **The STL containers are divided into three major categories**
- **sequence containers**
 - **associative containers**
 - **container adapters**



Containers Types and Examples

Standard Library container class	Description
<i>Sequence containers</i>	
vector	Rapid insertions and deletions at back. Direct access to any element.
deque	Rapid insertions and deletions at front or back. Direct access to any element.
list	Doubly linked list, rapid insertion and deletion anywhere.
<i>Associative containers</i>	
set	Rapid lookup, no duplicates allowed.
multiset	Rapid lookup, duplicates allowed.
map	One-to-one mapping, no duplicates allowed, rapid key-based lookup.
multimap	One-to-many mapping, duplicates allowed, rapid key-based lookup.
<i>Container adapters</i>	
stack	Last-in, first-out (LIFO).
queue	First-in, first-out (FIFO).
priority_queue	Highest-priority element is always the first element out.



Containers Types

- The **sequence containers** represent linear data structures
 - vectors and linked lists.
- The **associative containers** are nonlinear containers
 - locate elements stored in the containers quickly
 - store sets of values or **key/value pairs**.
- Stacks and queues actually are **constrained versions of sequential containers**.
 - STL implements stacks and queues as **container adapters**
 - enable a program to view a *sequential container* in a *constrained manner*.



Containers' Common Member Functions

- **Most STL containers provide similar functionality.**
- **Many generic operations, such as member function size, apply to all containers**
 - **other operations apply to subsets of similar containers.**
 - **encourages extensibility of the STL with new classes.**



Introduction to Iterators

- **Iterators have many similarities to pointers**
 - point to first-class container elements.
- **Certain iterator operations are uniform across containers.**
- **For example, the dereferencing operator (*) dereferences an iterator**
 - get the element to which it points.
- **The ++ operation on an iterator moves it to the container's next element**



Iterators

- STL first-class containers provide member functions **begin** and **end**.
- Function **begin** returns an iterator pointing to the first element of the container.
- Function **end** returns an iterator pointing to the first element past the end of the container (an element that doesn't exist).



Iterators

- Iterator **i** points to a particular element
 - **++i** points to the “next” element
 - ***i** refers to the element pointed to by **i**
- The iterator resulting from **end** is typically used in an equality or inequality comparison
 - determine whether the “moving iterator” (**i** in this case) has reached the end of the container.
- An object of type **iterator** refers to a container element that can be modified.
- An object of type **const_iterator** refers to a container element that cannot be modified.



Introduction to Algorithms

- STL algorithms can be used generically across a variety of containers.
- STL provides many algorithms to manipulate containers.
 - **inserting, deleting, searching, sorting** etc.
- The algorithms operate on container elements only indirectly through iterators.
- Many algorithms operate on sequences of elements defined by pairs of iterators
 - one pointing to the first element of the sequence
 - one pointing to one element past the last element



Introduction to Algorithms

- Algorithms often return iterators that indicate the results of the algorithms.
- Algorithm **find**
 - locates an element and returns an iterator to that element.
 - If the element is not found, **find** returns the “one past the end” iterator.
- The **find** algorithm can be used with any first-class STL container.
- Some algorithms demand powerful iterators; e.g., **sort** demands random-access iterators.



Vector

- **Provides an alternative to the built in array.**
- **A vector is self grown.**
- **Use It instead of the built in array!**



Defining a new vector

Syntax: vector<of what>

For example :

- vector<int>** - vector of integers.
- vector<string>** - vector of strings.
- vector<int * >** - vector of pointers to integers.
- vector<Shape>** - vector of Shape objects. Shape is a user defined class.



The vector Container

➤ Constructors

`vector<T> v, // empty vector`

`v1(100), // 100 elements of type T`

`v2(100, val), // 100 copies of val`

**`v3(fp_ptr,lp_ptr); // contains copies of
// elements in memory
// locations fp_ptr to lp_ptr`**



vector Operations

- **Information about a vector's contents**
 - **v.size()**
 - **v.empty()**
 - **v.capacity()**//expand by doubling its size
 - **v.reserve()**//grow its capacity to para
- **Adding, removing, accessing elements**
 - **v.push_back()**
 - **v.pop_back()**
 - **v.front()**//return a reference to v's first item
 - **v.back()**



Using Vector

- **#include <vector>**

- **Two ways to use the vector type:**
 - 1. Array style.**
 - 2. STL style**



Using vector – Array Style

We mimic the use of built-in array.

```
void simple_example()  
{  
    const int N = 10;  
    vector<int> ivec(N);  
    for (int i=0; i < 10; ++i)  
        cin >> ivec[i];  
  
    int ia[N];  
    for ( int j = 0; j < N; ++j)  
        ia[j] = ivec[j];  
}
```



Using vector – STL style

We define an empty vector
vector<string> svec;

we insert elements into the vector using the method push_back.

```
string word;  
while ( cin >> word ) //the number of words is unlimited.  
{  
    svec.push_back(word);  
}
```



Insertion

void push_back(const T& x);

Inserts an element with value x at the end of the controlled sequence.

svec.push_back(str);



Size

unsigned int size();

Returns the length of the controlled sequence (how many items it contains).

unsigned int size = svec.size();



Class Exercise 1

Write a program that read integers from the user, sorts them, and print the result.



Solving the problem

- **Easy way to read input.**
- **A “place” to store the input**
- **A way to sort the stored input.**



Using STL

```
int main()  
{  
    int input;  
    vector<int> ivec;  
  
    /* rest of code */  
}
```



STL - Input

```
while ( cin >> input )  
    ivec.push_back(input);
```



STL - Sorting

```
sort(ivec.begin(), ivec.end());
```

Sort Prototype:

```
void sort(Iterator first, Iterator last);
```



STL - Output

```
for ( int i = 0; i < ivec.size(); ++i )  
    cout << ivec[i] << " ";  
cout << endl;
```

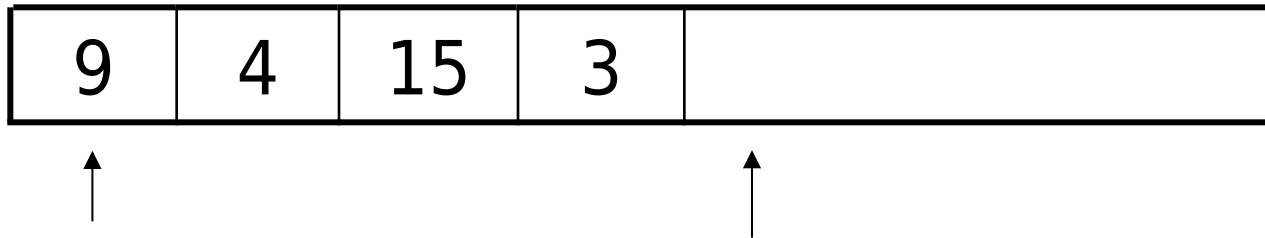
Or (more recommended)

```
vector<int>::iterator it;  
for ( it = ivec.begin(); it != ivec.end(); ++it )  
    cout << *it << " ";  
cout << endl;
```

Iterators

- **Given a vector which has had values placed in the first 4 locations:**

```
vector<int> v
```



- **`v.begin()` will return the iterator value for the first slot,**
- **`v.end()` for the next empty slot**



STL - Include files

```
#include <iostream> // I/O  
#include <vector>   // container  
#include <algorithm> // sorting  
  
//using namespace std;
```



Putting it all together

```
int main() {  
    int input;  
    vector<int> ivec;  
  
    // input  
    while (cin >> input )  
        ivec.push_back(input);  
  
    // sorting  
    sort(ivec.begin(), ivec.end());  
  
    // output  
    vector<int>::iterator it;  
    for ( it = ivec.begin();  
        it != ivec.end(); ++it ) {  
        cout << *it << " ";  
    }  
    cout << endl;  
  
    return 0;  
}
```



Operations on vector

- iterator begin();
- iterator end();
- bool empty();
- void push_back(const T& x);
- iterator erase(iterator it);
- iterator erase(iterator first, iterator last);
- void clear();
-



vector

- **Dynamic array of variables, struct or objects.**
- **Insert data at the end.**

```
#include <iostream>
#include <vector>
using namespace std;
int main () {
    vector<int> v; // Template!
    for (int i=1; i<=10 ; i++)
        v.push_back(i);
    v.erase (v.begin()+5);
    v.erase (v.begin(),v.begin()+3);
    v.pop_back();
    for (int i=0; i<v.size(); i++)
        cout << v[i] << " ";
    return 0;
}
```

4	5	7	8	9
---	---	---	---	---



Using iterator

```
#include <iostream>
#include <vector>
using namespace std;

int main () {
    vector<int> v1 (3,100);
    vector<int>::iterator it;
    it = v1.begin();
    it = v1.insert (it,200);
    v1.insert (it,2,300);
    it = v1.begin();
```

```
    vector<int> v2 (2,150);
    v1.insert( it+3, v2.begin(), v2.end());
```

```
    int a1 [] = { 501,502,503 };
    v1.insert (v1.begin(), a1, a1+3);
```

```
    for (it=v1.begin();it<v1.end(); it++)
        cout << *it << " ";
    cout << endl;
```

```
    return 0;
```

```
}
```

501 502 503 300 300 200 150 150 100 100 100



Exercise

Vector I

For a dynamic array $A=\{a_0, a_1, \dots\}$ of integers, perform a sequence of the following operations;

- `pushBack(x)`: add element x at the end of A
- `randomAccess(p)`: print element a_p
- `popBack(x)`: delete the last element of A

A is a 0-orign array and it is empty in the initial state.

Input

The input is given in the following format.

Q

Query₁

Query₂

Each query query_i is given by **0** x or **1** p or **2**

Where the first digits 0,1 and 2 represent pushback, randomAccess and popBack operations respectively.

randomAccess and popback operations will not be given for an empty array.



Exercise

Output

For each randomAccess, print a_p in a line.

Constraints

- $1 \leq q \leq 200,000$
- $0 \leq p < \text{the size of } A$
- $-1,000,000,000 \leq x \leq 1,000,000,000$

Sample Input

```
8
0 1
0 2
0 3
2
0 4
1 0
1 1
1 2
```

Sample Output

```
1
2
4
```



Exercise

Vector II

For n dynamic arrays A_i ($i=0,1,\dots,n-1$), perform a sequence of the following operations;

- `pushback(t,x)`: Add element x at the end of A_t .
- `dump(t)`: print all elements in A_t .
- `clear(t)`: clear A_t if A_t is empty, do nothing.

A_i is a 0-origin array and it is empty in the initial state.

Input

The input is given in the following format.

Q

Query₁

Query₂

Each query query _{i} is given by **0 t x** or **1 t** or **2 t**

Where the first digits 0, 1 and 2 represent pushback, dump and clear operations respectively.



Exercise

Constraints

- $1 \leq n \leq 1,000$
- $1 \leq q \leq 500,000$
- $-1,000,000,000 \leq x \leq 1,000,000,000$
- The total number of elements printed by dump operations do not exceed 500,000

Sample Input

```
3 13
0 0 1
0 0 2
0 0 3
0 1 -1
0 2 4
0 2 5
1 0
1 1
1 2
2 1
1 0
1 1
1 2
```

Sample Output

```
1 2 3
-1
4 5
1 2 3

4 5
```